

The Speed of Numbers

A team investigation into representation and latency on the ESP32

Course: FCA
Platform: ESP32 Dev Board + Arduino IDE (nothing else needed)
Work mode: In your team
Duration:
Team name: _____
Members: _____
Due date: _____

1 The big question

Every number inside a computer has a *representation*: 8, 16, 32 or 64-bit integers, float, double, fixed-point, or even text characters.

Does the way a number is represented change how long the processor takes to work with it? By how much? And why?

Nobody will hand you the answers. Your team will design small experiments on the ESP32, measure what really happens, and explain it. You only need the board and a USB cable.

2 How your team will work

The cycle: Predict → Measure → Explain

For every experiment:

1. **Predict.** Discuss as a team and write down what you expect, and why, *before* running any code. Do not change it afterwards.
2. **Measure.** Build the experiment and collect numbers.
3. **Explain.** Compare the result with your prediction. If you were wrong, that is the most interesting part: find out why.

Team roles

Rotate these roles for every experiment so everyone does each job at least once.

Builder writes and uploads the code.

Measurer runs the tests, repeats them, and records the numbers.

Skeptic challenges every result: “Are we sure? Could the compiler have changed this? Did we measure the loop instead of the operation?”

Scribe writes the experiment record (Section 6).

3 Using an LLM, the way we do in class

You are encouraged to use an LLM (ChatGPT, Claude, Gemini, or similar) as a **thinking partner**, just as we do in class. Use it to understand ideas, not to do the work for you.

Good ways to use it

- Ask it to **explain** a concept: “What does `volatile` do in C?”
- Ask it to **check your reasoning**: “We think `int8_t` will be faster because it is smaller. What might we be missing?”
- Ask it to **explain an error message** you got from the Arduino IDE.
- Ask it for a **hint**, not the full answer: “Give me a hint, not the code.”
- Ask it **why** a result might have happened, then test its explanation.

Rules

1. **The board is the judge, not the LLM.** An LLM can sound confident and still be wrong, especially about exact timings or ESP32 hardware details. If it makes a claim you can test, test it.
2. **Understand everything you submit.** Any member of your team may be asked to explain any line of code or any conclusion in your report.
3. **Keep an LLM log.** For every experiment, record the important prompts you used, what the LLM said, and whether your measurements agreed with it.

Try this at least once

Ask an LLM to *predict* one of your results before you measure it, such as “How many cycles does a `double` division take on an ESP32?” Then measure it yourself. Record whether it was right, and discuss why an LLM might get hardware-specific numbers wrong.

4 Your toolbox

You need a way to measure very short times. Here is the one piece of code you are given:

```
uint32_t t0 = ESP.getCycleCount();  
// ... the thing you want to time ...  
uint32_t t1 = ESP.getCycleCount();  
uint32_t cycles = t1 - t0;
```

Everything else, you build yourselves. Work through these questions as a team before you start the experiments. They will save you a lot of confusion later.

1. The ESP32 runs at 240 MHz. How long is one clock cycle?
2. A single addition takes a few nanoseconds. Reading the cycle counter also takes some cycles. So if you time just *one* addition, what are you really measuring? How could you fix that?
3. If you put the operation inside a loop, the loop itself costs time too. How can you find out what the loop costs, so you can remove it from your results?
4. If you run the same test several times, you get slightly different numbers. Should you report the average, the median, or the minimum? Justify your choice.
5. The compiler is clever. If it sees you calculate something and never use it, it may delete the calculation. If it knows the inputs in advance, it may calculate the answer while compiling. How can you stop it from doing this?

Clues

- Look up the C keyword `volatile`.
- Look up “loop overhead” and “baseline measurement”.
- A C `#define` macro can save you from copying the same timing code dozens of times.
- Set **Tools** → **CPU Frequency** to 240 MHz, and print `getCpuFrequencyMhz()` to confirm.

5 The experiments

Experiment 1: Build a stopwatch you can trust

Your mission

Build a reusable timing tool that reports the cost of an operation in cycles, with the loop overhead removed. Prove that it gives stable, believable results.

Questions to investigate

- How many cycles does a 32-bit integer addition take with your stopwatch?
- Do you get the same answer every time you reset the board?
- What happens if you remove `volatile`? What if you repeat the operation 10 times instead of 1000?

Clues

If your result is 0 or a strange negative number, something has been optimised away. The EN button on the board resets it so you can rerun your tests.

Experiment 2: Is smaller faster?

Your mission

Find out whether the width of an integer type changes how fast the ESP32 can add, multiply and divide with it.

Questions to investigate

- Is an 8-bit addition faster than a 32-bit addition? What about 16-bit?
- What happens with 64-bit integers? Which operation suffers most?
- Is signed division different from unsigned division?

Clues

- Types to test: `int8_t`, `int16_t`, `int32_t`, `int64_t`, `uint32_t`.
- Find out how wide the ESP32's registers are. Search for “Xtensa LX6”.
- After adding two 8-bit numbers, the result must still fit in 8 bits and keep its sign. Does the CPU need to do anything extra for that?
- How many registers does a 64-bit number need?

Experiment 3: float versus double

Your mission

Compare single-precision and double-precision floating point, and discover what the ESP32's floating-point hardware can and cannot do.

Questions to investigate

- How do `float` and `double` compare for add, multiply, divide and square root?
- Is floating-point division as fast as multiplication?
- Is `x * 0.5` the same speed as `x * 0.5f` when `x` is a `float`? Why or why not?
- How does the fastest `float` operation compare with a 32-bit integer addition?

Clues

- Look up what an FPU is, and find out whether the ESP32's FPU is single or double precision.
- If the hardware cannot do an operation, something else must. Search for “software floating point emulation”.
- In C, what is the type of the literal `0.5`? What about `0.5f`?
- `sqrt()` and `sqrtf()` are different functions.

Experiment 4: Are bit tricks really faster?

Your mission

Test the popular claim that shifts and masks are faster than multiply, divide and remainder. Find out when it is true, when it is false, and when it gives the *wrong answer*.

Questions to investigate

- Is `x / 8` slower than `x >> 3`? Is `x % 16` slower than `x & 15`?
- Does the answer change if the 8 is stored in a `volatile` variable instead of written directly in the code?
- What are `-9 / 8` and `-9 >> 3`? Are they equal?

Clues

- The compiler may already be doing the trick for you. How could you check?
- Write `-9` in 32-bit two's complement **by hand**, then shift it.
- Look up how C rounds integer division of negative numbers, and how an arithmetic right shift rounds.
- `Serial.println(x, BIN)` prints a number in binary.

Experiment 5: Invent your own fractions

Your mission

Represent fractional numbers using only integers (fixed-point), implement add, multiply and divide, and compare them with `float` for both speed and accuracy.

Questions to investigate

- How would you store 3.75 in the Q16.16 format? And -1.25 ?
- What is the smallest and the largest number Q16.16 can hold?
- Is fixed-point multiplication faster or slower than `float`? More or less accurate?

Clues

- Q16.16 means 16 bits for the whole part and 16 bits for the fraction, inside one 32-bit integer. What do you multiply a real number by to convert it?
- When you multiply two Q16.16 numbers, where does the binary point end up? You will need to shift the result.
- Multiplying two 32-bit numbers can produce a result that does not fit in 32 bits. Try it with real numbers and see what goes wrong. What type could hold the intermediate result?
- Use a `double` calculation as your “true” answer when measuring error.

Experiment 6: The hidden cost of printing**Your mission**

Converting a number into text is also a change of representation. Measure how much it costs, and compare it with the arithmetic you measured earlier.

Questions to investigate

- How many cycles does it take to turn an integer into text? A float?
- How long does it take to *send* one character over Serial at 115200 baud? Calculate it first, then measure it.
- Does printing a number in decimal, hexadecimal or binary take different amounts of time? Why?
- How many integer additions could the CPU do while one character is being sent?

Clues

- `snprintf()` converts to text in memory without sending anything, so it measures only the conversion.
- Look up how many bits are sent per character on a serial link (think about start and stop bits).
- `Serial.flush()` waits until all the output has actually been sent. Sending takes milliseconds, so `micros()` is fine here.
- Count the characters, including the invisible ones that `println` adds.

Final challenge: Meet a latency budget**Your mission**

A sensor must be sampled 10 000 times per second. Your team must choose the best number representation for processing each sample, and prove your choice with data.

The scenario. For each sample, the system must:

1. read a 12-bit value (`raw`, from 0 to 4095) from the ADC;
2. convert it to millivolts: $\text{mV} = \text{raw} \times 3300/4095$;
3. smooth it with a moving average of the last 8 samples.

You do not need a real sensor. You can call `analogRead()` on an unconnected ADC pin such as GPIO 34: the value will just be noise, but the *time* it takes is real. For testing your arithmetic, generate your own `raw` values in code.

Questions to investigate

- How many cycles does the system have for each sample at 10 kHz?
- How long does `analogRead()` take on its own?
- Build the conversion and filter three ways: using `double`, using `float`, and using only integers. How fast is each one?
- Over every possible `raw` value (0 to 4095), what is the biggest error of the `float` and integer versions compared with `double`?
- Which version would you ship? Would your answer change at 50 kHz or 100 kHz?

Clues

- For the integer version, can you find a constant K so that $\text{mV} \approx (\text{raw} \times K) \gg 16$? How would you calculate K ?
- How can you divide by 8 without using division?
- `random(0, 4096)` generates test values. A simple `for` loop can test every value from 0 to 4095.
- The ADC can only tell apart steps of about $3300/4095$ mV. Does an error smaller than that actually matter?

6 Writing up each experiment

Every experiment, including the final challenge, must be written up using the same structure. Write in your own words. Short and clear is better than long.

1. **Aim.** One or two sentences: what question were you trying to answer?
2. **Background.** What did you need to know first? Explain the key ideas briefly, such as register width, FPU or two's complement.
3. **Prediction.** What did your team expect, and why? (Written before measuring.)
4. **Method.** What exactly did you do? What code did you write? How many repetitions and trials? How did you remove the loop overhead? Someone else should be able to repeat your experiment from this section.
5. **Observations.** Your measurements in a clear table you design yourselves. Add a chart where it helps comparison.
6. **Analysis.** Explain your results. Where your prediction was wrong, explain why. Mention anything the Skeptic questioned and how you checked it.
7. **What we learnt.** Three to five sentences on what your team now understands that you did not before. Focus on ideas, not just numbers.
8. **LLM log.** The important prompts you used, what the LLM said, and whether the board agreed with it.

Your write-up must answer

- What did we expect, what actually happened, and why?
- What does this tell us about how the ESP32 represents and processes numbers?
- What would we do differently when writing real code, now that we know this?

7 What to submit

One submission per team, containing:

1. **Code:** your `.ino` file(s), with comments showing which experiment each part belongs to.
2. **Raw output:** the Serial Monitor output for each experiment, saved as text.
3. **Report (PDF):**
 - a write-up for every experiment, using the structure in Section 6;
 - one summary chart comparing the cost of every operation you measured;
 - your final challenge recommendation;
 - a **Biggest surprise** section: the result that was furthest from your prediction, and what it taught you;
 - a short **Team reflection**: who did which role in each experiment, and what you learnt about using an LLM to investigate something.

Exact cycle counts will differ between boards and software versions. What matters is the quality of your reasoning, not matching a particular number.

8 If you get stuck

Nothing appears in the Serial Monitor.

Check that the baud rate matches your code. Add a short delay at the start of `setup()`, and press EN after opening the monitor.

Upload fails with “Failed to connect”.

Hold the BOOT button while the IDE says “Connecting...”.

The board keeps restarting.

A test is probably running too long without a break. Look up the ESP32 watchdog timer.

A result looks impossible.

Good. That is the Skeptic’s job. Ask what the compiler might have done, and check.

Still stuck?

Ask a teammate, then an LLM (and verify what it says), then your instructor.

Predict honestly. Measure carefully. Explain everything.